

WF2026 会议主题教学笔记

赋能工程师与 Codex App

从代码补全到软件工程代理

Alexander Emiricos & Roman Huitt, OpenAI

原视频主题范围：00:28:37–00:48:09



核心结论：这段 keynote 的主线围绕工程师能力扩张展开：Codex 代码代理体系（Codex）帮助工程师把注意力从逐行输入转向目标拆解、验证、审查、部署和系统设计。Codex 应用（Codex App）的产品形态把聊天、可操作界面、开放代理栈和云端任务（cloud task）合成一个软件工程工作台。

证据范围：英文原始字幕为主证据，主题视频切片为图像证据，时间戳均使用原视频绝对时间。

原视频入口：<https://www.youtube.com/watch?v=htM02KMNZnk&t=1717s>

目录

阅读导引：先抓住这条主线	2
1 工程师的角色：从写代码回到解决问题	3
1.1 World's Fair：把未来做成可看见的公共物件	3
1.2 从媒体误解到工程责任：AI engineers are eating the world	3
1.3 本章小结	4
2 从补全到代理：工作循环如何变长	6
2.1 能力序列：从补全输入到承担目标	7
2.2 验证循环与 Codex 产品方向：从候选代码到可落地工作	7
2.3 本章小结	8
3 Codex App 的产品形态：聊天入口与可深入的协作界面	10
3.1 两个产品问题：代理能做什么，人该怎样协作	10
3.2 Chat 与 Hands on：先沟通，再按需深入	11
3.3 本章小结	11
4 开放代理栈：OpenAI 自己使用的层，也开放给开发者	13
4.1 分层开放的结论	13
4.2 Models 与 Responses API：把内部原语先放进 API	14
4.3 Codex Harness、App Server、App 与 Plugins：从循环到生态入口	15
4.4 本章小结	16
5 价值最大化：成本、速度、云端任务与工作方式重构	18
5.1 从 tokens 到工程产出：价值最大化的四个杠杆	18
5.2 成本与速度：更便宜的前沿能力，更快的方案探索	19
5.3 本地、云端与并行隔离：让代理自己选择合适环境	19
5.4 本章小结	20
结语：Codex App 的真正位置	22

阅读导引：先抓住这条主线

这段 keynote 的主线是：Codex 应用把代理能力放进工程师已经熟悉的工作循环中。读者可以把 Codex 应用理解为一个“软件工程工作台”：聊天是入口，任务委派是执行循环，图形界面让人类保留检查和介入能力，开放代理栈让生态工具接入，成本、速度和云端环境决定代理能否进入日常工作流。

学习目标

读完本文后，读者应能解释三件事：第一，演讲者如何把 AI 描述为扩大工程师能力，并降低简单替代叙事的解释力；第二，Codex App 相比传统代码补全，把工作循环延长到了哪些工程环节；第三，OpenAI 为什么同时强调开放代理栈、订阅复用、成本效率、速度和云端任务。

英文术语	中文解释	本文使用方式
Codex App	Codex 应用	指演讲中展示的面向软件工程协作的应用界面，包含聊天、任务视图、预览、审查与部署相关入口。
agent	智能体 / 代理	指能围绕目标调用工具、修改代码、运行检查并汇报结果的系统。本文在产品语境中多写作“代理”。
delegate a task	委派任务	指工程师把一个目标交给 Codex 推进，工作焦点从等待下一行代码补全转向完整任务推进。
Responses API	Responses API	指 OpenAI 提供给开发者的模型与工具调用接口；演讲将内部 Codex 需求与该 API 的原语演进联系起来。
Codex Harness	Codex Harness	指运行代理循环、工具调用与环境交互的底层执行层。
App Server	App Server	指把应用、登录、订阅、harness 控制和产品能力连接起来的一层。
Plugins	插件	指把浏览器、computer use、领域工具等能力接入代理应用的扩展机制。
TerminalBench 2.1	终端任务基准	演讲用它展示模型在终端式工程任务上的成本与效果主张。

1 工程师的角色：从写代码回到解决问题

本章先给出全文立场：Codex（OpenAI 在本次 keynote 中讨论的代码代理体系）要放大的，是工程师处理问题的半径。Alexander Emiricos 和 Roman Huitt 的开场把工程师放回完整位置：理解真实问题，吸收最新科学，把设计、品味、判断和想象力组合起来，做出别人真正能用的东西。

这一定义决定了后文的阅读方式。Codex 后续被展开为应用、开放代理栈、云端任务和价值最大化（value maxing）工具时，核心问题始终相同：它怎样帮助工程师更快进入问题本身，更可靠地把方案落到可检查的工程成果里。键盘输入是工程活动的一层执行动作，工程师的责任还包括目标选择、质量判断、风险边界和可用性。

本节核心判断

Codex 的教学入口是“扩大工程师的问题处理半径”：重复输入的占比下降，问题定义、方案设计、质量品味、风险判断和可用结果交付的占比上升。后续章节讨论的软件工程代理（software engineering agent），应被理解为工程师扩大行动半径的工具。

1.1 World’s Fair：把未来做成可看见的公共物件

演讲者先借用 World’s Fair（世界博览会）的类比来解释这场会议的气质。World’s Fair 的教育价值在于把遥远未来做成公共展品：原本听起来不可能的想法，变成可以看见、走近、讨论甚至相信的东西。对应到 WF2026，演讲者强调房间里有超过 7,000 名 AI engineers，他们正在日常使用并推动这项技术。

这个类比服务于一个更大的判断：未来的工程形态正在被现场的人建造。开发者、研究者、产品工程师和工具制造者通过具体系统、具体界面、具体工作流，让抽象趋势逐步显形。对读者来说，这句话的含义很实际：理解 Codex 需要同时看模型生成代码的能力，以及它怎样改变工程师发现问题、组织任务、检查结果和交付价值的方式。

World’s Fair 类比的教学作用

World’s Fair 的关键是把未来做成可走进、可比较、可讨论的公共物件。类比到软件工程，新的 AI 能力需要通过产品、接口、工作流和实际项目被展示出来；只有当能力变成可操作的物件，工程师才有条件判断它的边界、风险和用途。

1.2 从媒体误解到工程责任：AI engineers are eating the world

演讲者进入核心论点时，先点名一个常见媒体误解：“engineers are going away”。这个误解的推理是，coding 会被抽象掉，所以工程师终将失去必要性。它的问题在于把 engineering 压缩成打字写代码这一层执行动作，忽略了工程师还要定义问题、判断质量、组织系统、处理风险和交付可用结果。

开场最强的修辞来自两句英文源短语的连续改写：先是“software ate the world”，随后是“AI engineers are eating the world”。这句话接在媒体误解之后，形成了反驳链：软件已经成为世界运行的底层结构，AI 又开始改写软件生产方式，真正决定能力如何抵达普通人的，是能够把 AI 能力变成可靠系统的工程师。

因此，本节对“工程师”的定义是一组能力组合，覆盖岗位标签之外的责任。演讲者给出的组合很具体：最新科学提供可用能力，设计让能力进入人的场景，品味决定什么值得保留，判断负责质量与风险，想象力负责把尚未成形的可能性转化为产品雏形。AI 让代码生产的局部动作变快，工程师的高层职责随之变得更突出。

这种判断也解释了为什么演讲者说现在是成为工程师的好时机。他给出的具体节奏证据是：过去大约每 15 个月发布一个新模型，现在大约每 6 周就会有一次模型发布节奏，并且他提到上周已经发布了 5.6 series 的 preview。这组数字说明工程师面对的基础设施正在快速变化。

若工程被理解为问题求解，那么模型发布节奏加快、工具能力上升、代码输入成本下降，都会把注意力推向更上游的问题：应该解决什么，为什么值得解决，怎样验证方案，怎样让别人信任并使用它。Codex 在全文中的角色，就是围绕这些问题给工程师提供更大的行动半径。

1.3 本章小结

本章建立全文的第一层答案：Codex 的意义要放在工程师角色的重心迁移中理解。开场的 World's Fair 类比告诉读者，AI 工程的未来需要通过可见、可检查、可讨论的物件被建造出来；“AI engineers are eating the world”则把责任落到人身上：工程师把 AI 能力接入真实问题，并承担设计、判断、质量和交付责任。这里的证据链顺序很关键：先有“工程师会消失”的媒体误解，再有工程责任的反驳，随后用 15 个月到 6 周的模型发布节奏说明技术底座正在加速。后续章节会继续说明，当代理能够补全、修改、测试、协作和云端执行时，这个“回到解决问题”的定义如何变成具体工作流。这个定义也是后文判断所有产品细节的标尺。

源画面证据

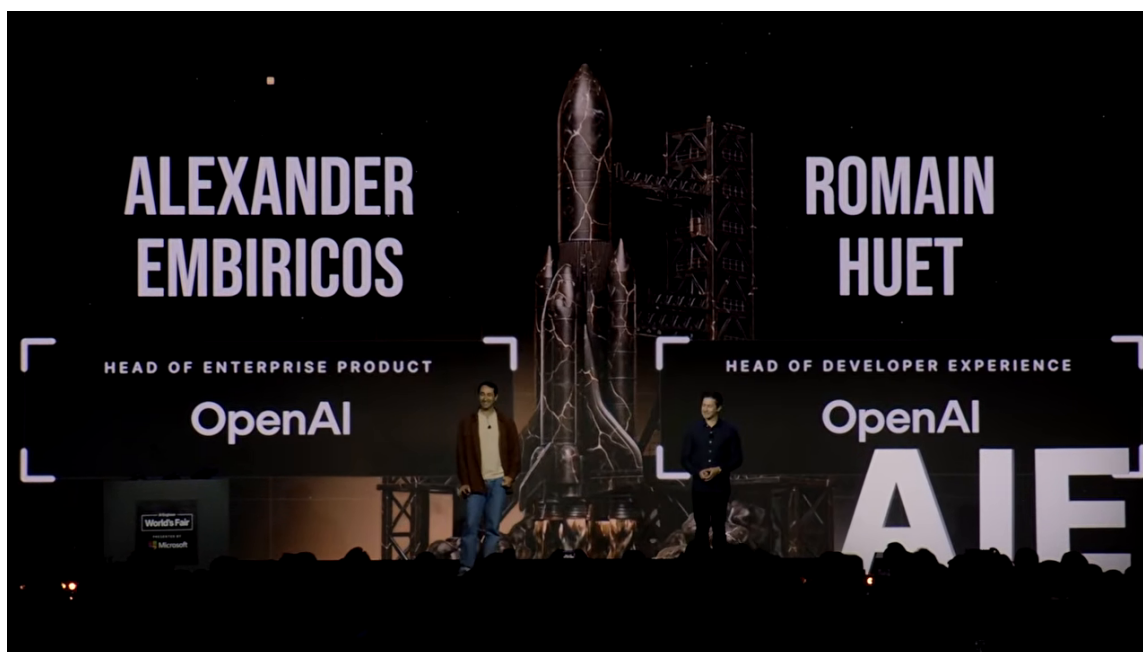


图 1：开场画面给出两位演讲者和 OpenAI 身份，本主题从 OpenAI 的 Codex 产品实践出发。（原视频绝对时间：00:28:42–00:29:04）

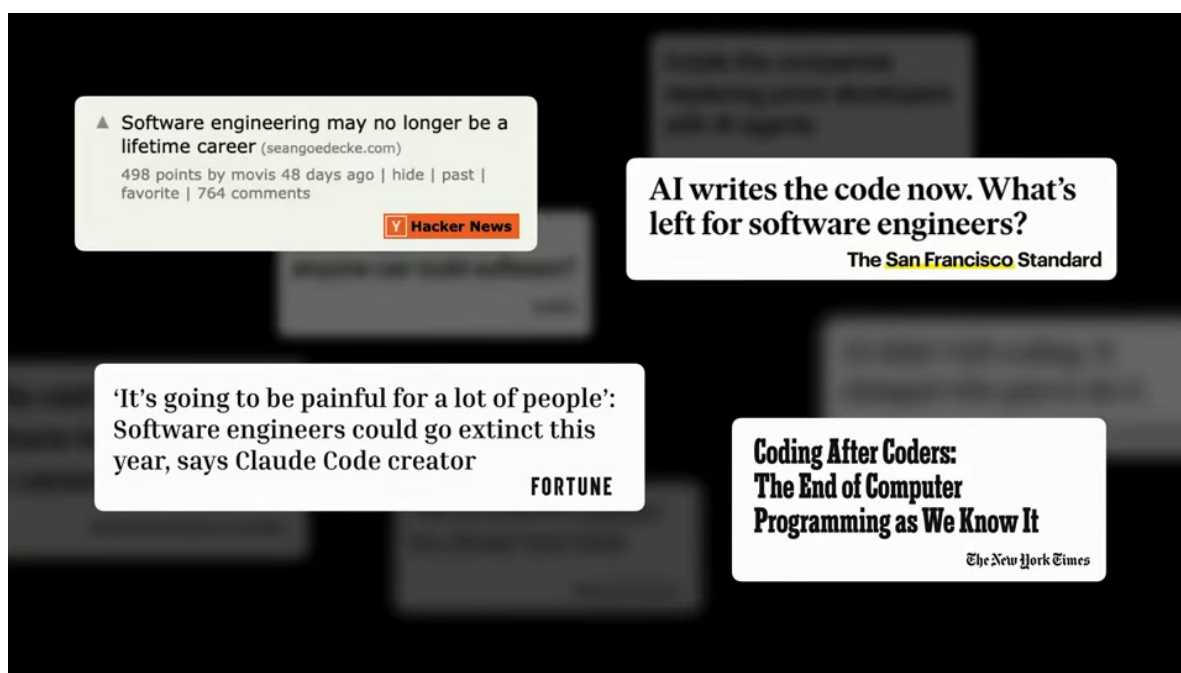


图 2：新闻标题集合把“工程师是否会消失”的误解显性化，为后续反驳提供问题背景。（原视频绝对时间：00:30:06–00:30:23）

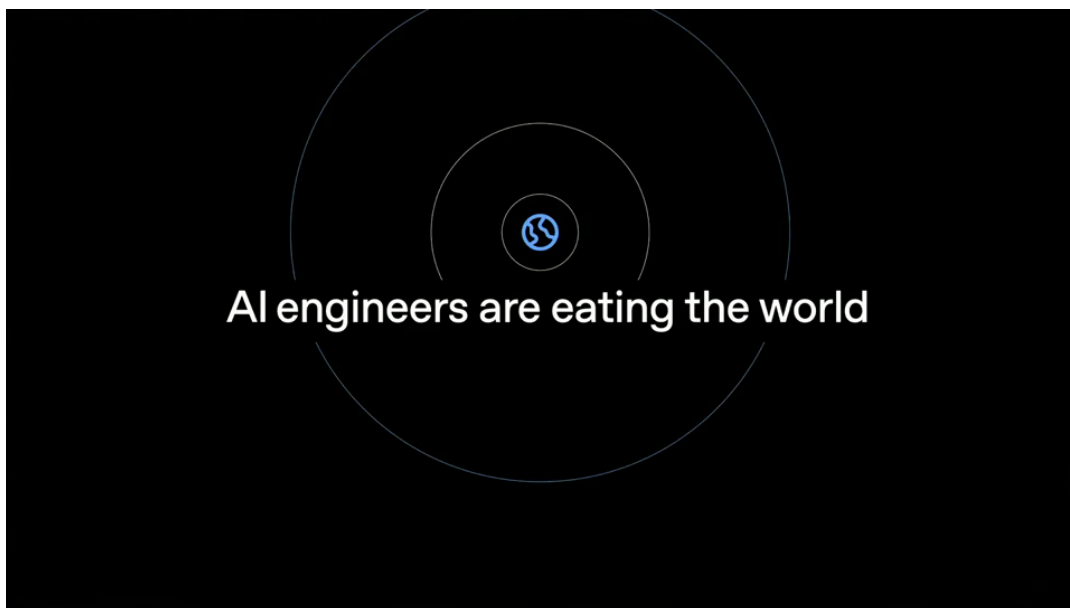


图 3: “AI engineers are eating the world”把演讲的反驳压缩成一句口号：核心变化是工程师能力扩张。（原视频绝对时间：00:30:20–00:30:35）

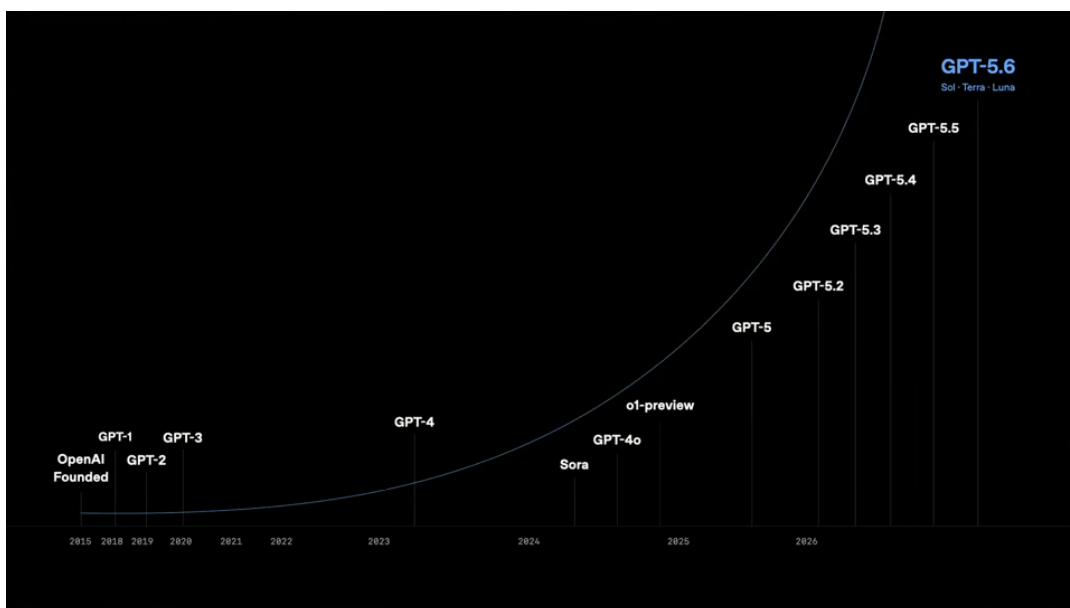


图 4: 源幻灯片用 GPT-5.6 曲线表达模型持续进展；本文只把该曲线作为 keynote 内部主张来支撑工程角色变化讨论。（原视频绝对时间：00:31:10–00:31:39）

2 从补全到代理：工作循环如何变长

本章结论先行：这段演讲把软件 AI 的价值重心从“生成一段代码”推进到“在工程循环里完成一项工作”。补全、inline prediction 和 Command-K 风格编辑都能缩短输入时间；真正改变工程手感的是编码代理（coding agent）开始进入构建、测试与检查（test/check）、差异视图（diff）、代码审查（code review）和部署这一整条链路。演讲者用 Dev Day 2024

与 2025 的现场 demo 对比说明：同样是模型写代码，是否能运行代码并验证结果，会直接决定工程师对结果的信心。

常见误解：把代码生成等同于已落地工程工作

模型能生成代码，只说明它完成了“提出候选实现”这一步。工程工作还需要构建、测试与检查、阅读差异视图、代码审查，以及必要时的部署和回滚判断。缺少这些环节时，工程师仍要靠手工验证承担风险；有了这些环节，编码代理才开始从“写代码工具”进入“可交付工作循环”。

2.1 能力序列：从补全输入到承担目标

演讲者先给出一个压缩的能力年表：很长一段时间里，开发者拥有的是 completion，也就是模型根据上下文补全符号、语句或局部代码。随后出现 inline prediction，模型开始预测下一步编辑，并把建议放在编辑器里更贴近开发者的工作流。再往后，Command-K 风格的编辑让开发者可以用自然语言要求模型“改一段代码”。

这一序列的关键限制在 00:31:53–00:31:59 被明确说出：模型可以被要求改代码，但当时“wouldn’t test the work”。换句话说，早期能力已经能减少敲键盘和局部查找成本，却没有进入工程闭环。它更像一名会给出草稿的助手，能节省起步时间，仍把正确性证明、运行反馈和风险承担留给人。

随后，模型开始测试自己的工作；再往后，模型开始承担“long hard goals until they’re done”。这里的“长目标”值得拆开理解：它包含任务目标、代码修改、运行反馈、失败修正、再次验证和最终交付。编码代理在这个阶段的工作单位变长了，从一个符号、一个代码块、一次编辑，扩展成一段可追踪、可检查、能回到人类审查环节的工程任务。

2.2 验证循环与 Codex 产品方向：从候选代码到可落地工作

Roman Huitt 用 Dev Day 2024 的例子解释这条分界线。字幕记录显示，他当时使用预览模型从零构建一个迷你无人机界面¹。演示能大概率运行，但模型无法实际运行代码，也无法验证自己的工作；所以现场 demo 依赖人的经验、准备和运气。

到 Dev Day 2025，他说自己已经有足够信心让模型控制完整的摄像机系统和灯光系统，因为模型能够测试自己的工作。这个对比没有把“测试”当作附加功能处理；它把测试呈现为工程信任的来源。生成代码只是提出方案，测试与检查让模型看到运行结果，失败时回到修改步骤，成功时把更可信的差异视图交给人审。

因此，构建与测试循环可以被理解为一條工程传送带：

目标/请求 -> 代码修改 -> 构建与测试 -> 检查 diff -> 代码审查 -> review/deploy

¹该处字幕把模型名转写为“01 in preview”；可据 Dev Day 2024 语境理解为预览模型称呼，不能推出新的模型事实。

这条链路的每一环都有不同责任。目标/请求定义“为什么要做”；代码修改给出候选实现；构建与测试提供机器可观察反馈；差异视图让人能看到实际改变；代码审查把团队规范、安全性、可维护性和产品判断带回流程；review/deploy 则决定这项工作是否真正进入用户可感知的系统。

在 00:33:21–00:33:27，演讲者把 Codex 代码代理体系（Codex）描述成能够完成用户自己电脑上可以做的任务，并点名他们今年最喜欢交付的能力包括 Codex 应用（Codex App）、goal mode 和 remote。这些能力的共同点是改变“做工作”的感觉，因此代理参与的范围开始覆盖编码前、编码中和编码后。

编码前，代理需要理解工作为什么值得做。演讲者说，如果能把代理连接到“why it has to be done”，代理就能开始承担更多工作。这对应产品和工程中的需求澄清、约束识别、方案选择和原型试验。编码中，代理执行修改、运行检查、迭代修复。编码后，代理连接 review and deploy，让输出进入代码审查、部署或交接（handoff）流程，从而“land much more work”。

这也解释了为什么该段最后把速度和决策质量联系起来。更快的循环让团队能原型化更多想法，并把更多时间花在用户身上。这里的因果链很具体：编码代理缩短单次实现成本，测试与检查降低盲目信任风险，代码审查和部署连接交付路径，产品团队因此能用更多真实反馈选择该构建什么。

2.3 本章小结

本章的核心判断是：软件 AI 的工作单位正在变长。completion 解决局部输入成本，inline prediction 改善编辑流，Command-K 风格编辑让自然语言进入修改环节；自测试模型和长目标代理把工程循环扩展到运行反馈、差异检查、代码审查和交付判断。Dev Day 2024/2025 的对比说明，验证能力是从“生成候选代码”走向“形成可审查工程工作”的关键。Codex App、goal mode 和 remote 在演讲中被作为产品证据提出，说明 OpenAI 正把代理连接到编码前的意图、编码中的执行、编码后的 review/deploy，使工程师能更快试验、更稳验证，并围绕用户反馈作出更好的构建决策。

源画面证据



图 5: “Delegate a task”画面把工作循环从补全推进到修改、运行测试和验证结果。（原视频绝对时间：00:31:53–00:32:18）

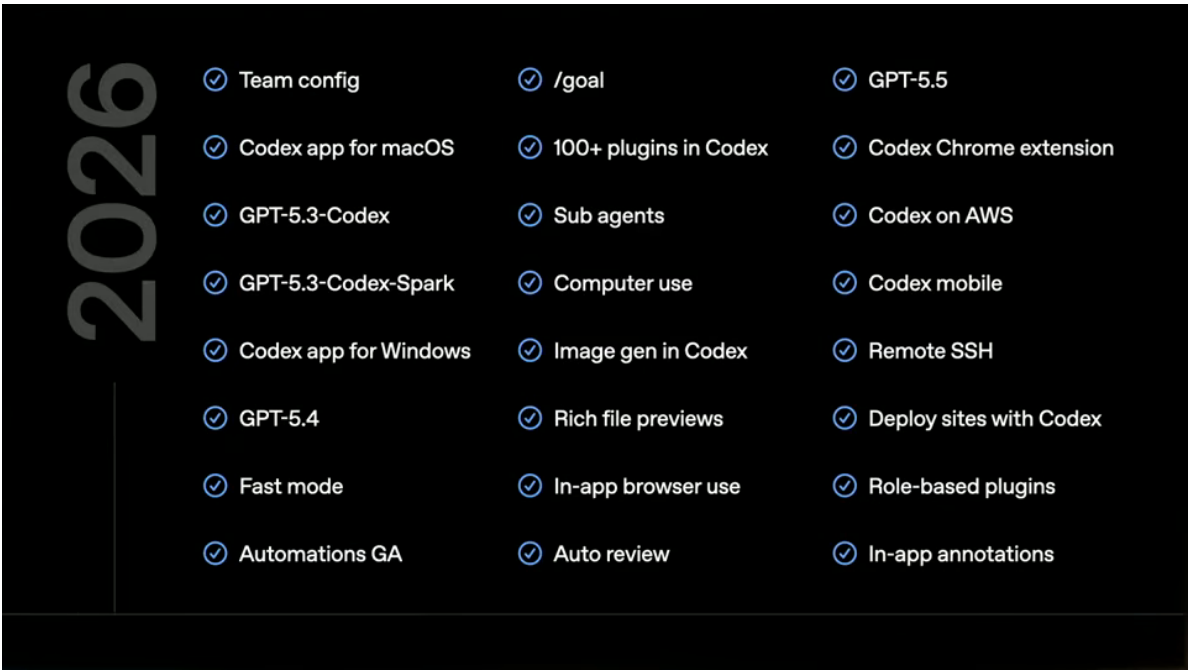


图 6: 2026 年发货清单展示 Codex App、goal mode、remote 等能力，证明产品方向已经从编辑器内提示扩展到完整工作流。（原视频绝对时间：00:33:04–00:33:27）

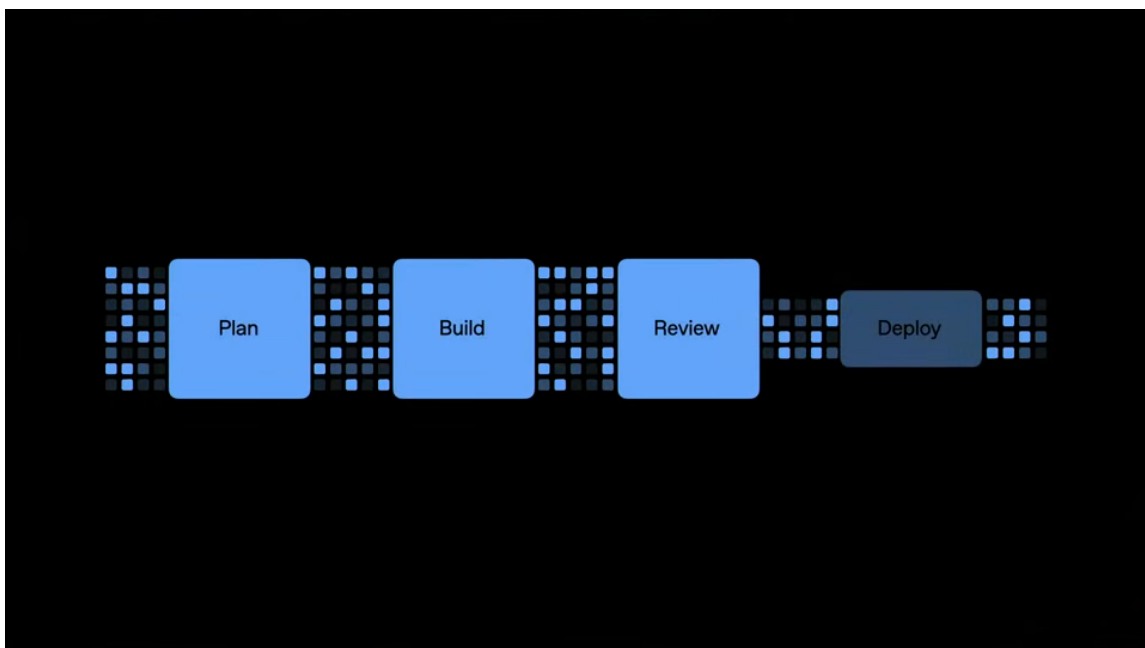


图 7: Plan、Build、Review、Deploy 四个动词说明 Codex App 试图覆盖软件交付闭环。(原视频绝对时间: 00:33:21–00:34:08)

3 Codex App 的产品形态：聊天入口与可深入的协作界面

3.1 两个产品问题：代理能做什么，人该怎样协作

本章的结论先给出：Codex 应用（Codex App）的产品形态服务于一个具体目标，即让工程师在代理能力增强时仍然掌握方向、判断和细节控制。演讲者先把问题拉回 OpenAI 的使命表述：“AGI that benefits all of humanity”。在这个框架下，产品问题被拆成两类：第一，代理被允许连接哪些工具、环境和行动能力；第二，人应该通过什么界面与这些代理共同完成工作。

这里的关键含义是，代理能力本身无法自动决定产品形态。模型可以处理越来越多中等长度的计算机任务，甚至在平均任务上超过单个讲者的自评表现；产品仍然需要回答授权边界、运行位置、协作方式和可检查性。演讲者把这些问题合并到一个产品目标上：最大化赋能工程师。换句话说，Codex 应用的界面设计要让工程师把更多任务交给代理，同时保留必要的检查、指挥和修正入口。

这一段也给后文的云端任务、开放代理栈和价值最大化埋下约束：如果代理逐步连接到更多系统，产品就需要明确它能做什么、在哪里运行、由谁批准关键动作、人在什么时候进入细节。缺少这些约束时，强模型会变成一个能力很强却难以管理的黑箱；约束设计清楚时，它才更像一个可以协作、可以复盘、可以纳入工程流程的队友。

3.2 Chat 与 Hands on: 先沟通, 再按需深入

演讲者把协作界面压缩成两个互补模式: **Chat** 和 **Hands on**。Chat 是最低摩擦的入口, 适合提出目标、解释背景、询问方案和交代任务; Hands on 是深入入口, 适合检查代理正在做什么、指向具体对象、调整局部细节, 或者直接自己动手改动。两者共同构成 Codex 应用的核心产品判断: 让用户先用自然语言启动协作, 再在需要时进入可操作界面。

团队与工作台的类比

演讲者用团队协作来解释这个界面形态。与队友合作时, 大多数协调发生在对话里: 说明目标、同步进展、讨论取舍。只有当某个细节需要共同判断时, 人才会走到工作台前, 一起看具体材料、拆解问题、调整实现。Codex 应用试图把这个模式迁移到人机协作: Chat 对应日常沟通, Hands on 对应走到工作台前共同处理细节。

这个类比能解释为什么单纯的命令入口不够完整。任务刚开始时, 工程师往往还在描述目标、补充背景、调整优先级; 这一阶段更像和队友交谈。任务进入实现细节时, 工程师又需要看到具体对象: 界面元素、代码差异、运行结果、局部变更点。Codex 应用把这两个阶段放在同一产品面里, 使工程师可以从一句话请求切换到具体对象级别的指挥。

演示里的世界杯比分例子看似轻量, 实际承担了产品说明功能: 用户可以从聊天式请求开始, 随后指向屏幕上的具体元素, 要求代理修改某一处, 也可以自己完成修改。这说明 Hands on 的作用超过结果展示; 它更接近一个可共同操作的工作面, 让人把抽象意图落到界面对象和代码对象上。

隐藏细节会削弱掌控感

产品设计的一种常见错误, 是把代理结果包装成过于平滑的黑箱, 让用户只能接受或拒绝最终输出。演讲者强调的 “feeling of mastery” 指向相反要求: 工程师需要能够进入细节、拆开问题、指向局部、检查代理动作, 并在必要时自己接管。掌控感属于工程工作可信落地的前提, 不能被降级为界面装饰。

这也解释了终端 (terminal)、Vim 和 Emacs 的采用阻力故事。演讲者提到, 早期有人直接表示不会离开 terminal、Vim 或 Emacs; 团队内部也质疑为什么需要再建一个应用, 因为工程师已经热爱 CLI 和 IDE。这个故事主要说明产品边界; 编辑器文化只是背景。CLI 适合命令和线性对话, IDE 适合从代码对象出发, Codex 应用试图承载一种更宽的协作顺序: 先聊天确认任务, 再在需要时深入细节。IDE 从代码开始, CLI 容易停留在文字交互, Codex 应用要覆盖从目标沟通到细节操控的连续过程。

3.3 本章小结

本章要保留的核心判断是: Codex 应用的产品形态来自两个问题的合并。代理侧问题是它能连接什么、能在哪里运行、能替人完成哪些行动; 人侧问题是工程师如何发起任务、观察过程、修正方向并保留掌控感。演讲者给出的答案是 Chat 加 Hands on 的互补结构。

对工程团队而言，这个结构的直接启发是：成功的代理产品同时依赖模型能力与协作界面。自然语言入口降低启动成本，可深入的操作界面降低黑箱风险。工程师由此可以像带队友一样带代理：大多数时候通过对话分配和校准工作，关键时刻进入工作台，检查、指向、修改、接管。

产品设计复盘

本章的设计复盘可以压缩成一句话：Codex 应用把“说清目标”和“看清细节”放到同一个协作空间。若只有聊天，用户很难检查代理实际改了什么；若只有代码对象，用户又要在任务刚开始时承担太多定位成本。Chat 加 Hands on 的组合让任务从意图进入对象，再从对象回到决策。

源画面证据

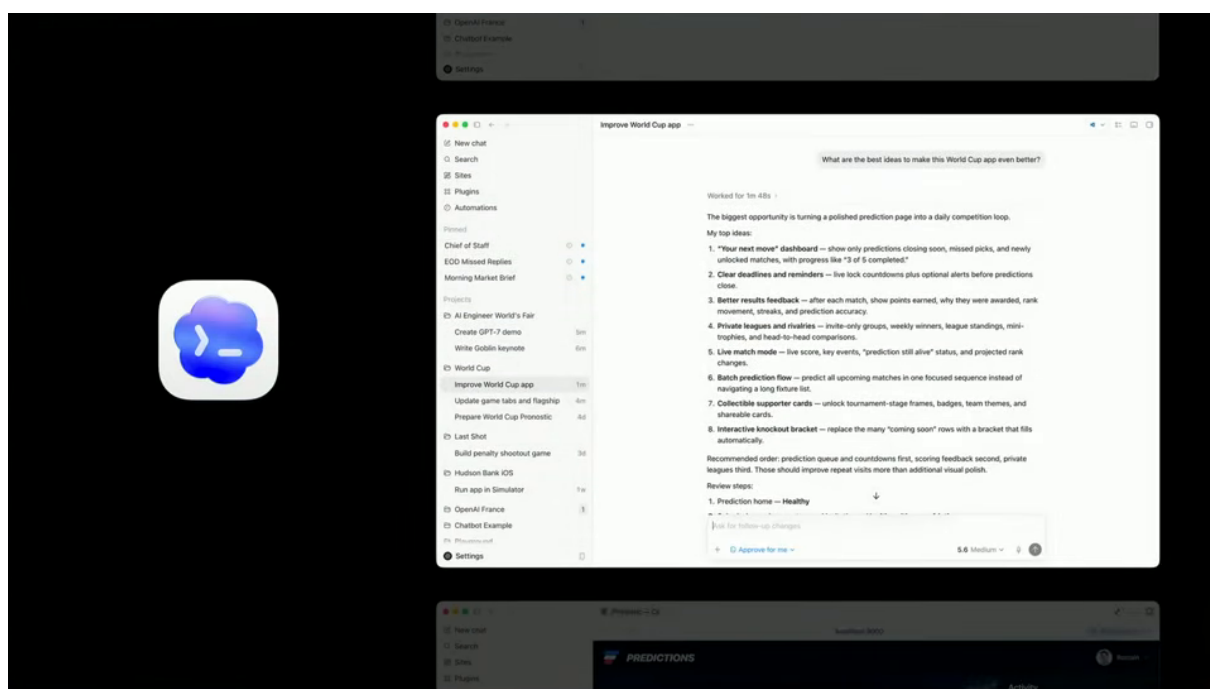


图 8: Codex App 的聊天与任务视图并置，体现“先自然语言协作，再检查具体任务状态”的产品结构。（原视频绝对时间：00:37:12-00:37:45）

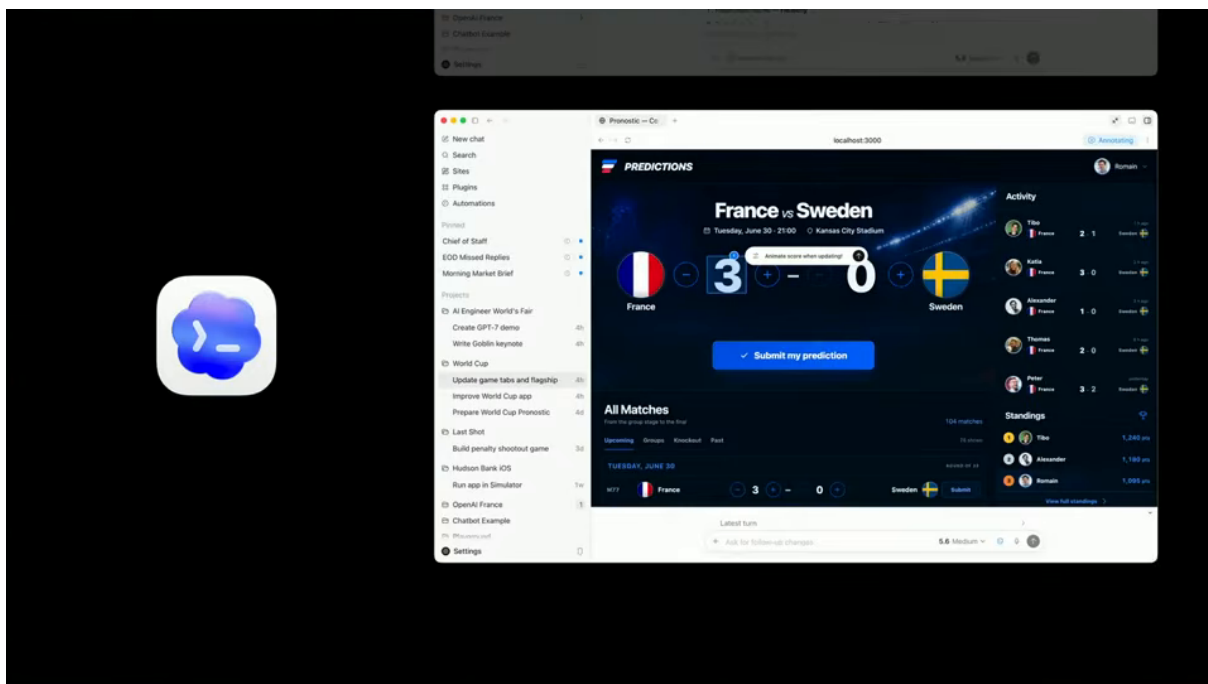


图 9: World Cup 示例展示应用内预览与互动反馈，说明用户可以在代理执行结果上继续检查和介入。（原视频绝对时间：00:37:45–00:38:05）

4 开放代理栈：OpenAI 自己使用的层，也开放给开发者

本章的结论很具体：演讲者把 Codex 体系描述成一组可复用的开放层，OpenAI 用这些层构建自己的 Codex 应用（Codex App），开发者也可以在同一组层之上搭建自己的代理产品、插件和工作流。这个主张的重点在于“层”与“复用”：底部是 Models 和 Responses API，中间是 Codex Harness 与 App Server，上层是 App 与 Plugins；每一层都承担不同责任，也都提供给外部开发者使用。

4.1 分层开放的结论

Roman Huitt 在这一段先说明背景：他们在产品界面和模型层上推进很快，同时也从社区实践中学习。用户会把 Codex 用到 OpenAI 团队未必提前想到的场景里，这些实验反过来影响 OpenAI 接下来要做什么。由此推出本节的核心前提：如果 Codex 只能由 OpenAI 自己改进，它就无法吸收这种生态反馈；因此 Codex 被设计成“anyone can build on”的分层栈。

可以把这套栈理解成一栋楼的剖面图。Models 像地基，Responses API 像标准化管线，Codex Harness 像可替换、可检查的工作循环，App Server 像把循环接入产品与订阅体系的服务层，App 像面向人的操作界面，Plugins 像给不同任务和工具加上的外接能力。读者需要抓住的是责任边界：每层解决一个问题，跨层组合以后才变成完整的代理体验。

层名	本层回答的问题	演讲中的开放含义
Models	代理用什么智能能力理解任务、调用工具、处理代码。	OpenAI 自己的产品和开发者都使用同一类模型能力。
Responses API	开发者通过什么 API 使用模型原语。	Codex 需要的新原语会优先进入 Responses API，例如长上下文任务的 compaction。
Codex Harness	代理循环如何运行、如何调用工具、如何进入环境。	该层开源，可检查、fork、改造，也可保留代理循环并更换模型选择。
App Server	应用如何控制 harness，并接入登录、订阅和产品形态。	OpenAI 称 VS Code extension 和 Codex 应用也走这条路径。
App	用户通过什么界面协作、检查和操控代理。	OpenAI 希望应用层创新不受自身想法限制。
Plugins	新能力如何以扩展点接入代理应用。	in-app browser、browser use、computer use 和若干领域插件被作为例子。

表 2：开放代理栈的责任划分。层名保留源幻灯片英文，解释为教学重绘。

同一套内部与外部系统

这一段最重要的主张是：OpenAI 交付给开发者的层，也是它自己构建 Codex 的层。演讲者在 00:42:58–00:43:16 明确强调，OpenAI 在每一层实际使用交给开发者的东西；开发者 fork harness、测试模型能力边界、提出新用法，都会成为 OpenAI 学习和改进系统的来源。

4.2 Models 与 Responses API：把内部原语先放进 API

底层从 Models 开始。前文已经讨论模型进展速度，这里进一步说明模型属于产品系统的一部分；开发者通过 Responses API 使用这些模型，OpenAI 构建 Codex 应用时也经由同一 API 使用同一类模型。这个说法把 Responses API 放在“内部产品”和“外部开发者”之间：它承担演讲者声称自身也依赖的产品原语层责任。

Responses API 的具体例子是 compaction。长时间运行的代理任务会积累大量本地上下文（local context）：项目文件、工具输出、用户指令、历史决策、错误和修复记录都会进入任务状态。上下文一长，代理若每次都完整携带全部信息，成本、延迟和注意力分配都会恶化。compaction 的作用是把长上下文压缩成更可继续使用的任务状态，保留后续行动所需的关键信息，让代理可以继续推进长期任务。

为什么 compaction 属于 API 原语

把 compaction 放进 Responses API，等于把“长任务如何继续”这类基础能力从某个单一应用中抽出来。开发者构建自己的代理时，面对的也是长上下文、本地上下文和阶段性总结问题；如果 API 提供同一原语，生态中的代理可以在相似的任务压力下复用 OpenAI 内部产品验证过的机制。

这个例子能支持的结论是“同层原语复用”：Codex 最近需要 compaction，OpenAI 会把它构建进 API，使开发者代理能使用同样原语。它没有展开所有内部实现细节，也没有说明每个开发者代理默认自动获得完整的 Codex 产品行为。读者应把重点放在产品原则上：内部产品用到的长任务能力，会被整理成外部开发者也能调用的基础原语。

4.3 Codex Harness、App Server、App 与 Plugins：从循环到生态入口

中间层首先是 Codex Harness。演讲者强调它是 open source，开发者可以 inspect、fork、adapt。这说明开放的对象覆盖代理循环本身：如何接收任务、如何调用工具、如何在环境中导航、如何把模型能力落到可执行步骤上。OpenAI 模型是 harness 的默认选择；字幕同时说明模型没有 hardcoded，开发者可以使用 open model，并保留相同的 agent loop。

AGENTS.md 在这一层承担“共享指令文件”的角色。演讲者说他们没有为 Codex 重造一个只能自己使用的文件格式；他们选择了一个其他 agents 也能使用的名称。AGENTS.md 可以理解为代理可读的项目指令文件：它把本地上下文中的长期约束、项目规则、工具偏好和安全边界放到代理能读取的位置。对于开发者，这相当于把“这个项目如何工作”的隐性知识沉淀成可复用入口。

演讲者还把 Codex Harness 放进 post-training environment。这里的意思是：模型在训练或后训练过程中接触的工具调用和环境导航方式，与实际开源 harness 的形态保持关联。这个主张的重要性在于减少“训练时学一套、部署时用另一套”的落差。若模型在更接近真实 harness 的环境中学习，工具调用、环境探索和代理循环的行为更容易与开发者实际拿到的层一致。

再往上一层是 App Server。演讲者给出的动机是：当开发者想把 harness 放进应用，并让用户用现有 Codex 订阅登录时，需要统一控制 harness 的方式；OpenAI 自己构建 VS Code extension 和 Codex 应用时也遇到同一问题，于是构建并开源 App Server。这里的关键边界是：App Server 被描述为 OpenAI 自己产品使用的路径，同时也开放给外部应用；它在这段叙事中处于主路径位置。同一段源画面里的推文还提到 ChatGPT 账号可用于一组其他工具，这说明账号与订阅复用是生态叙事的一部分，同时仍不等于这些外部工具具备相同稳定承诺、权限模型或发布状态。

Codex Monitor 例子说明 App Server 的生态用途：Tuma（X 上的 Dimilian）在 Codex 应用发布前，用 App Server 构建过自己的 Codex 原生应用，后来加入 OpenAI 团队并构建 Codex for iOS。这个例子强调服务层可以支撑原生应用和团队内部产品，也提醒读者把

稳定性、权限模型和发布状态放到具体应用里分别判断。

上层是 App 与 Plugins。App 层关注人如何使用代理，Plugins 层关注能力如何扩展。演讲者列举 in-app browser、plugins、browser use、computer use，说明这些能力通过同一 extension points 构建，并且可以被开发者看到和借鉴。对于工程团队，这相当于把“让代理多会一种动作”的问题，转化为插件边界、权限边界和 UI 反馈边界的问题。

演讲者最后还提到 OpenCode、Pi、Droid、OpenClaw、Xcode、JetBrains 等名称，作为“现有订阅可以在越来越多地方使用”的生态例子。这些名称的作用是展示生态覆盖范围，并不自动说明每个产品当前的具体集成深度、商业关系或上线状态。这里真正服务于教学的结论是：订阅、应用、IDE 和插件都被放进同一个开放栈叙事里，OpenAI 希望开发者在不同入口复用同一底层能力。

开放栈的使用顺序

阅读这套开放栈时，可以按三层做判断：Responses API 负责模型原语，Codex Harness 负责可执行代理循环，App Server、App 与 Plugins 负责把循环接入产品、订阅、界面和具体工具。团队引入这类能力时，权限、稳定性、审核和集成深度都应回到对应层中处理。

4.4 本章小结

这一章把 Codex 体系从单个产品界面拆成六层：Models 提供智能能力，Responses API 暴露模型原语，Codex Harness 承载可执行的代理循环，App Server 把 harness 接入应用和订阅，App 提供协作入口，Plugins 扩展具体行动能力。它的教学价值在于让读者看清“开放”对应一组具体层级、边界和复用点。

从工程实践角度看，这套开放代理栈给开发者三类抓手。第一，开发者可以从 API 层复用 compaction 等长任务原语。第二，开发者可以从 Codex Harness 层检查、fork、改造代理循环，并通过 AGENTS.md 把项目指令固化为共享上下文。第三，开发者可以从 App Server、App 和 Plugins 层把代理接入自己的应用、订阅入口、浏览器能力、计算机使用能力或领域插件。演讲者的最终落点是：OpenAI 希望外部开发者在每一层推动能力边界，OpenAI 也通过这些实践学习和改进 Codex。

源画面证据

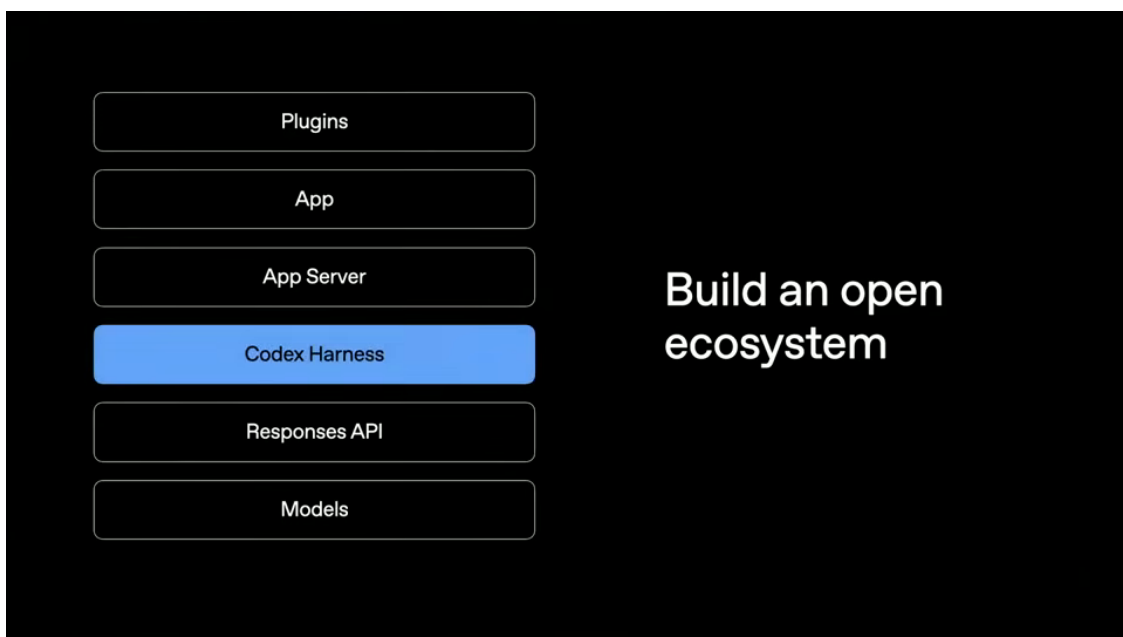


图 10: 六层开放代理栈把 Models、Responses API、Codex Harness、App Server、App、Plugins 的责任边界放在同一张图中。(原视频绝对时间: 00:39:35–00:40:02)

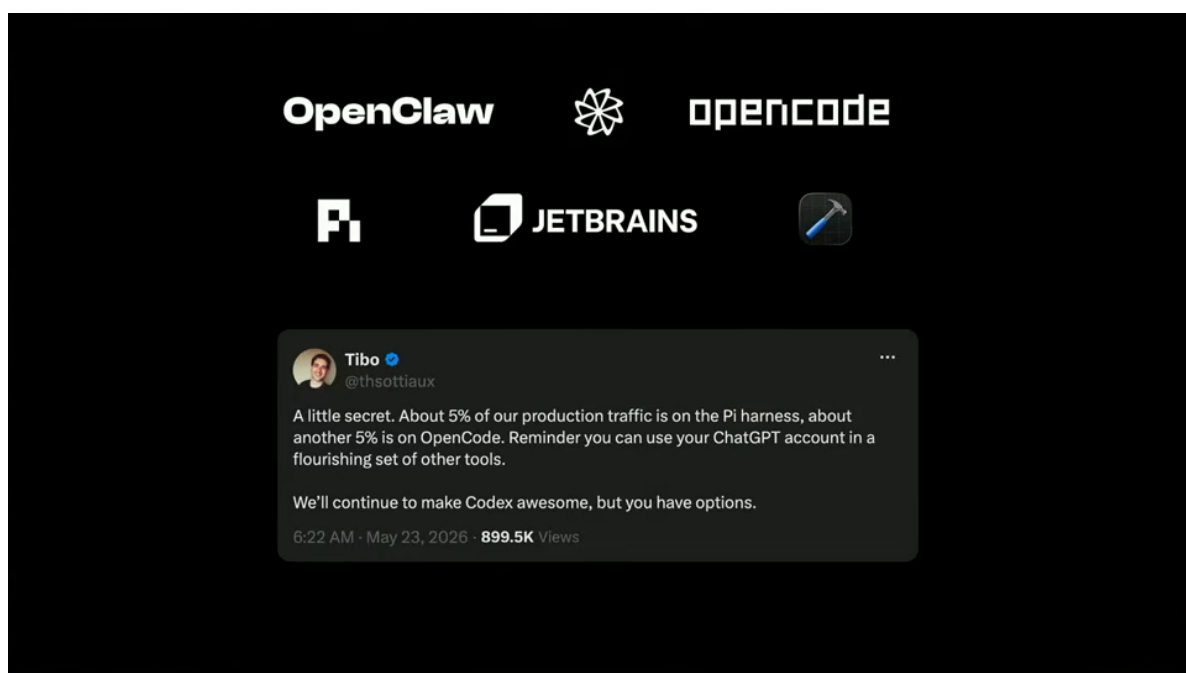


图 11: 生态工具和订阅复用画面说明 OpenAI 希望开发者在 Codex 外也能使用同一账号与相邻工具; 源推文同时提到 ChatGPT 账号可用于其他工具。(原视频绝对时间: 00:40:40–00:41:08)

5 价值最大化：成本、速度、云端任务与工作方式重构

本章的结论是：演讲最后把 Codex 的技术栈问题推进到工程管理问题。前四章解释了工程师角色、代理循环、Codex 应用（Codex App）和开放代理栈；这一章回答更落地的问题：团队怎样从代理中得到可用价值。Roman Huitt 使用“value maxing”这个说法，把焦点从 tokens 消耗转向有用工程产出：代理需要在可承担的成本、足够快的速度、合适的运行环境和可审查的交付链路里工作，才真正扩大工程师处理问题的半径。

价值最大化的工作定义

价值最大化（value maxing）指用代理产生更多可验证、可交接、可落地的工程结果。tokens 是成本输入之一，真正的目标是让软件工程代理（software engineering agent）在任务理解、代码修改、测试与检查、差异视图（diff）、代码审查（code review）和交付之间形成稳定闭环。

5.1 从 tokens 到工程产出：价值最大化的四个杠杆

演讲者说，与工程负责人讨论代理时，主题会从“生成更多内容”转向怎样从代理中提取价值。这个判断很重要：若团队只盯着输出长度，代理会被当作更快的文本机器；若团队盯着可落地的工程产出，代理就必须进入真实工作链路，承担方案探索、代码生成、检查、审查准备和交接（handoff）等动作。

可以把价值最大化拆成四个杠杆。成本决定团队能否把前沿模型用于大量任务；速度决定代理是否能试多条路径；环境决定任务能否离开本地机器持续运行；审查与部署链路决定代理产物能否变成团队愿意接受的变更。

杠杆	它改善什么	对工程团队的含义	风险边界
成本	让高能力模型能被更频繁地调用。	同一预算可以覆盖更多任务、更多尝试和更多团队成员。	团队仍要用自己的任务和预算做验证。
速度	缩短从任务到候选结果的等待时间。	代理可以并行尝试多种实现，再让工程师比较差异。	高速生成仍需要测试与检查、代码审查。
环境	让任务在本地任务（local task）与云端任务（cloud task）之间合理分配。	代理可以脱离笔记本开盖状态，在隔离环境中并行推进。	环境选择仍牵涉权限、依赖、数据边界和可复现性。
审查/部署	把代理输出接到拉取请求（pull request, PR）、diff、review 和 deploy。	人类保留判断权，代理产物进入团队已有交付秩序。	自动生成不等于自动合并，落地仍需可审查证据。

表 3：价值最大化的四个工程杠杆。该表是教学整理，源依据来自 00:43:52–00:48:09 的演讲总结。

成本、速度和可靠性需要一起看：能力图回答模型是否足够强，成本图回答调用是否可承担，速度数字回答交互是否快到支持多方案探索。这三类线索共同决定代理能否从少数展示任务进入日常工程任务。

5.2 成本与速度：更便宜的前沿能力，更快的方案探索

第一组杠杆是成本。演讲者先说工程团队希望拥有 frontier intelligence，也会挑选自己信任的评测来判断模型能力。随后他以 TerminalBench 2.1 为例展示柱状图：幻灯片把 GPT-5.6 Sol Ultra、GPT-5.6 Sol、GPT-5.6 Terra、GPT-5.6 Luna 等模型放到同一图上比较，并给出若干分数与价格说法²。

成本效率的教学意义是理解“可用规模”：某个价格只是一条来源证据。昂贵的前沿能力像一台只能供少数专家排队使用的实验设备；成本下降以后，它才可能进入日常工程流程，覆盖重构、测试补齐、文档更新、bug 复现和代码审查准备等大量中等粒度任务。换句话说，成本杠杆把代理从少数关键任务的展示品，推向团队日常生产力工具。

第二组杠杆是速度。演讲者接着展示“GPT-5.6 Sol on Cerebras”和“750 tokens/sec”，并用“十秒左右写出一个 substantial PR”来帮助听众形成直觉。这里的 PR 指拉取请求：它是一个可审查的代码变更单元，通常包含代码差异、解释、测试结果和讨论记录。这个类比的作用是说明速度达到某个量级后，交互方式会改变；它不能被扩展成任意任务都能十秒完成的保证。

速度的真正价值在于并行探索。一个慢代理通常只能给出一条候选路径，工程师随后判断是否继续；一个足够快的代理可以同时尝试五六种办法，再把不同 diff、测试结果和权衡交给人类比较。这个工作方式更接近“让多名同事各自尝试方案，然后进行设计评审”；等待单一聊天答案的占比会下降。速度因此改变了工程师的决策面：人类从亲自写第一版，转向设定目标、筛选候选、审查证据和决定落地。

5.3 本地、云端与并行隔离：让代理自己选择合适环境

第三组杠杆是运行环境。演讲者用办公室里“大家把笔记本开着，好让 agents 继续工作”的场景，说明本地任务的现实限制：任务跑在用户电脑上时，机器是否开盖、网络是否稳定、本地依赖是否可用、权限是否合适，都会影响代理能否继续推进。这个场景很具体，因为它把代理从抽象智能拉回到普通工程基础设施。

Codex Cloud 被演讲者放在这个问题的回答中。他说团队从第一次 major launch 开始就在瞄准这件事，并提到 Codex Cloud 将迎来 major upgrades。这里的教学重点是“隔离并行”：多个任务可以在各自环境里运行，避免相互污染依赖、端口、文件状态和实验结果。对于工程团队，这相当于给代理分配多个可丢弃的工作台，每个工作台独立试一条路线，最后把可审查结果交回主线。

²这些模型、分数和价格来自本段 keynote 的现场幻灯片。团队选型时仍需要结合自己的任务、预算和评测方法重新验证。

演讲者进一步提出，未来不应该让用户手动纠结本地任务和云端任务的区分。更自然的交互是：用户在任何地方对代理说明目标，代理判断自己需要什么资源，再选择可用环境。这个设想把环境选择从用户心智负担变成代理规划的一部分。它也带来隐藏依赖：代理必须理解项目文件、权限、网络、密钥、数据边界和团队策略；若这些依赖没有被显式建模，云端并行只会把本地混乱复制到更多环境里。

Theo 的推文在演讲中作为趋势信号出现：多数开发者把 code agents 从笔记本转移出去，大约是六个月量级。演讲者的回应是节奏可能更快。这个片段把注意力指向计算位置的转移：当代理承担长期任务，计算位置会从“用户设备”转向“最适合任务的环境”。

5.4 本章小结

本节也是全篇的收束：Codex 的价值覆盖两层，一层是模型能参与写代码，另一层是它把工程师、代理、开放栈、运行环境和交付流程连成一个可控制的工作系统。第一章说明工程师仍负责定义问题和判断结果；第二章说明代理必须进入编辑、运行和检查循环；第三章说明 Codex 应用用聊天和 hands-on 协作保留人的掌控感；第四章说明 OpenAI 把 Models、Responses API、Codex Harness、App Server、App 与 Plugins 组织成可复用层；第五章把这些能力放进成本、速度、环境和 review/deploy 的价值框架里。

演讲最后跳过 live demo，把下一位嘉宾的上台解释为一个更大的信号：这是重新想象“如何工作、如何构建”的时刻。对本文读者而言，最实用的抽象是“工程师协调多个代理”：工程师提出目标和边界，代理在本地或云端选择合适环境，多个隔离任务探索不同方案，系统保留上下文与交接记录，最终通过 PR、diff、测试与检查和代码审查进入团队交付链路。

落到工程团队，三点最关键。第一，价值最大化关注有用工程输出，tokens 只是成本侧输入。第二，速度和成本的组合会改变工作形态：更便宜的前沿能力让代理进入日常任务，更快的响应让多方案并行成为常规选择。第三，云端任务和本地任务的边界会逐步退到后台，关键问题转向权限、隔离、上下文、审查和交接。若这些边界被设计清楚，Codex 这类软件工程代理才能从“会生成代码的工具”走向“可被工程团队长期委派和审查的协作者”。团队真正要管理的变量，是任务目标、运行环境、验证证据和交付路径是否足够明确。

源画面证据

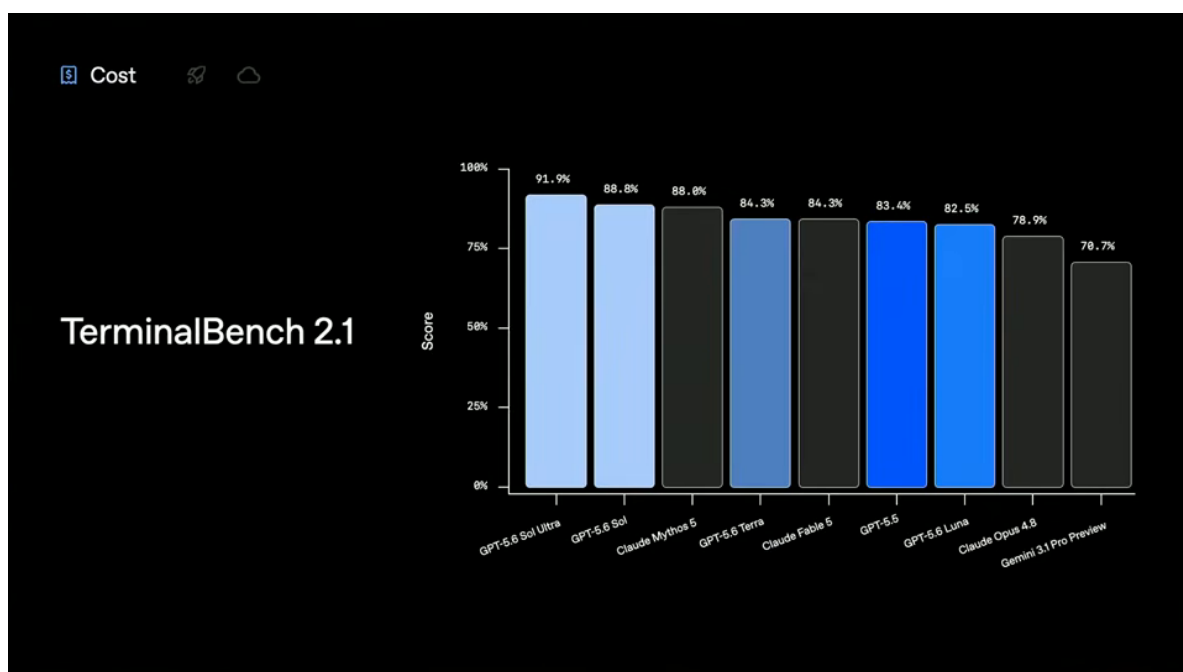


图 12: TerminalBench 2.1 成本图展示演讲中的效率主张：同等工程任务下，模型能力需要同时考虑效果与调用成本。（原视频绝对时间：00:44:18–00:45:04）

这张图的读法应保持克制：柱状图给出的是 keynote 现场用于说明成本效率的来源证据，教学价值在于提示工程团队同时评估效果、价格和调用频率。若只看最高分，团队会忽略预算约束；若只看价格，团队又会忽略能否完成真实工程任务。

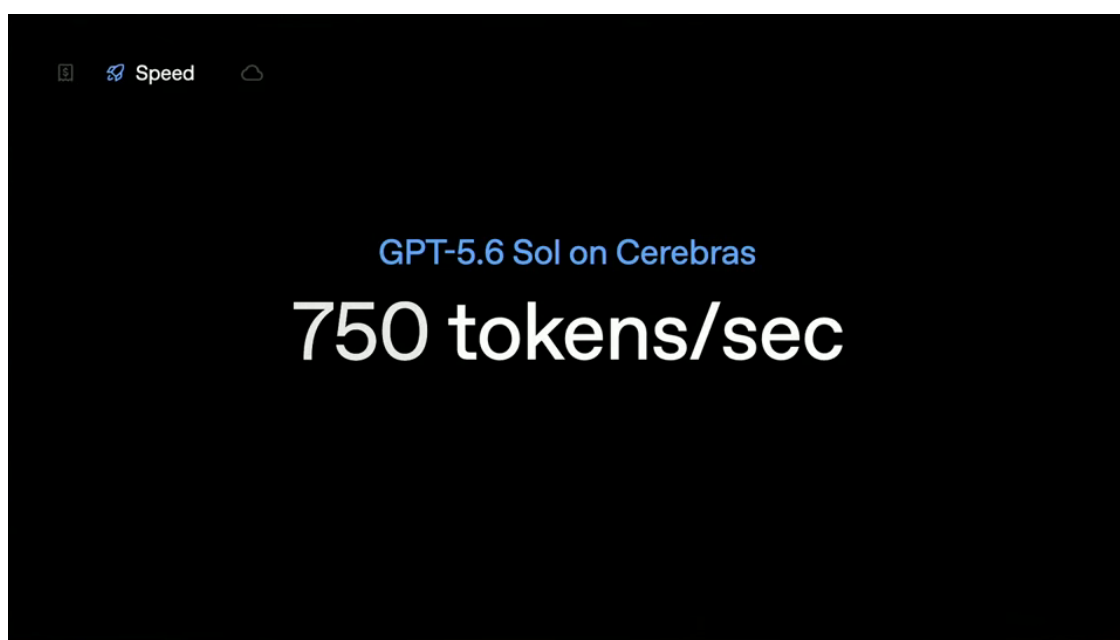


图 13: 源幻灯片写明“GPT-5.6 Sol on Cerebras”与“750 tokens/sec”；该数字在讲稿中作为演讲者速度主张，用于说明快速候选方案会改变工程迭代方式。（原视频绝对时间：00:45:27–00:46:11）



图 14: 演讲者用 Cloud 标签和合上的笔记本表达 Codex Cloud 的产品想象：任务可在云端隔离环境中继续推进。（原视频绝对时间：00:46:32–00:46:57）

结语：Codex App 的真正位置

本主题最终把 Codex App 定位为一种工程组织 workflows 中的代理协作入口。它把人类工程师的位置从“输入每一行”调整为“定义目标、设定约束、检查证据、决定合并和部署”。这个变化的风险也很清楚：如果任务边界、权限、测试、审查和可追溯证据缺失，代理会放大错误；如果这些工程约束被系统化，代理就会放大工程师的有效产出。

面向工程团队的检查清单

把本主题落到团队实践时，可以用四个问题做自检：任务目标是否足够清楚，代理能否理解“为什么要做”；运行环境是否明确，任务应放在本地任务还是云端任务中；验证证据是否完整，是否包含测试与检查、差异视图和代码审查；交接路径是否清楚，代理产物能否进入 PR、review/deploy 或其他团队已有交付秩序。

三条落地边界

第一，能力评估要同时看任务完成率、调用成本和响应速度。第二，代理并行会放大工程吞吐，也会放大权限、依赖、数据边界和上下文交接问题。第三，Codex App 的价值来自人和代理之间的可检查协作：人定义目标和质量边界，代理推进候选实现，系统保留证据，人再决定接收、修改、合并或部署。